

Programming Assignment 2
Nathan Lenahan
9/17/26

Analysis of the two GenAI Programs:

- a) The two GenAI programs used to generate the two programs were Claude and Gemini
- b) Both Claude and Gemini were accessed through their respective websites using Google Chrome

c) The following prompt was used for both GenAI programs:

CREATE THE FOLLOWING PROGRAM USING C CODE

OVERVIEW:

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

TEST FILE FORMAT:

Command: EMAIL <sender category>,<subject line>,<date>

Description of EMAIL command:

The emails in the CEO's Inbox should be placed in a queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

Command: NEXT

Description of NEXT command:

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

Command: READ

Description of READ command:

CEO has read email and dealt with it

Command: COUNT

Description of COUNT command:

Display current count of unread emails

The grader will test your code with a different test file. The test file will begin with the emails in the CEO's Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

SAMPLE TEST FILE:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

THE SAMPLE TEST FILE PROGRAM SHOULD DISPLAY THIS:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

d) The following is the generated code from both GenAI programs:

CLAUDE - Raw Generated Code

```
/* =====
 * email_priority.c
 *
 * Prioritizes a CEO's emails using a MaxHeap (array/list based
 * binary heap) implemented entirely from scratch -- no library
 * heap module is used.
 *
 * Priority rules:
 * 1. Sender category (higher category = read first):
 *   Boss      (5)
 *   Subordinate (4)
 *   Peer      (3)
 *   ImportantPerson (2)
 *   OtherPerson (1)
 * 2. Within the same category, the NEWEST date wins (read first).
 *
 * Commands read from stdin, one per line:
 * EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
 * NEXT
 * READ
 * COUNT
 * ===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_LINE    1024
#define MAX_SENDER  64
#define MAX_SUBJECT 512
#define MAX_DATE    16
#define INITIAL_CAPACITY 16

/* ----- */
```

```

* Email record stored in the heap.
* ----- */
typedef struct {
    char sender[MAX_SENDER]; /* sender category string, e.g. "Boss" */
    char subject[MAX_SUBJECT]; /* subject line */
    char date[MAX_DATE]; /* original date string MM-DD-YYYY */
    int categoryRank; /* precomputed numeric priority rank */
    long dateValue; /* precomputed YYYYMMDD numeric value */
} Email;

/* ----- *
* MaxHeap: dynamic array based binary heap of Email records.
* ----- */
typedef struct {
    Email *data; /* backing array (list) */
    int size; /* number of emails currently in heap */
    int capacity; /* allocated capacity of data array */
} MaxHeap;

/* =====
* Utility: trim trailing newline / carriage return from a line
* ===== */
static void trimNewline(char *s) {
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[len - 1] = '\0';
        len--;
    }
}

/* =====
* Map a sender category string to a numeric priority rank.
* Higher number = higher priority (read sooner).
* ===== */
static int categoryToRank(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0; /* unknown category -- lowest priority */
}

/* =====
* Convert a date string "MM-DD-YYYY" into a comparable numeric
* value of the form YYYYMMDD (bigger number = more recent date).
* ===== */
static long dateToValue(const char *date) {
    int month = 0, day = 0, year = 0;

```

```

    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0; /* malformed date -- treat as oldest possible */
    }
    return (long)year * 10000L + (long)month * 100L + (long)day;
}

/* =====
 * MaxHeap creation / memory management
 * ===== */
static void heapInit(MaxHeap *heap) {
    heap->capacity = INITIAL_CAPACITY;
    heap->size = 0;
    heap->data = (Email *)malloc(sizeof(Email) * heap->capacity);
    if (heap->data == NULL) {
        fprintf(stderr, "Error: could not allocate memory for heap.\n");
        exit(EXIT_FAILURE);
    }
}

static void heapFree(MaxHeap *heap) {
    free(heap->data);
    heap->data = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

/* Grow the backing array when the heap becomes full. */
static void heapGrow(MaxHeap *heap) {
    int newCapacity = heap->capacity * 2;
    Email *newData = (Email *)realloc(heap->data, sizeof(Email) * newCapacity);
    if (newData == NULL) {
        fprintf(stderr, "Error: could not grow heap memory.\n");
        exit(EXIT_FAILURE);
    }
    heap->data = newData;
    heap->capacity = newCapacity;
}

/* =====
 * Compare two emails for max-heap ordering.
 * Returns 1 if email 'a' has strictly HIGHER priority than 'b',
 * otherwise returns 0.
 * ===== */
static int hasHigherPriority(const Email *a, const Email *b) {
    if (a->categoryRank != b->categoryRank) {
        return a->categoryRank > b->categoryRank;
    }
    /* Same category: newer date (larger dateValue) wins. */
    return a->dateValue > b->dateValue;
}

```

```

}

static void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* =====
 * Sift the element at index 'idx' UP toward the root until the
 * max-heap property is restored (used after inserting).
 * ===== */
static void siftUp(MaxHeap *heap, int idx) {
    while (idx > 0) {
        int parent = (idx - 1) / 2;
        if (hasHigherPriority(&heap->data[idx], &heap->data[parent])) {
            swapEmails(&heap->data[idx], &heap->data[parent]);
            idx = parent;
        } else {
            break;
        }
    }
}

/* =====
 * Sift the element at index 'idx' DOWN toward the leaves until
 * the max-heap property is restored (used after removing root).
 * ===== */
static void siftDown(MaxHeap *heap, int idx) {
    while (1) {
        int left = 2 * idx + 1;
        int right = 2 * idx + 2;
        int largest = idx;

        if (left < heap->size && hasHigherPriority(&heap->data[left], &heap->data[largest])) {
            largest = left;
        }
        if (right < heap->size && hasHigherPriority(&heap->data[right], &heap->data[largest])) {
            largest = right;
        }
        if (largest == idx) {
            break;
        }
        swapEmails(&heap->data[idx], &heap->data[largest]);
        idx = largest;
    }
}

/* =====

```

```

* Insert a new email into the heap.
* ===== */
static void heapInsert(MaxHeap *heap, const char *sender, const char *subject, const char *date) {
    if (heap->size == heap->capacity) {
        heapGrow(heap);
    }

    Email *slot = &heap->data[heap->size];
    strncpy(slot->sender, sender, MAX_SENDER - 1);
    slot->sender[MAX_SENDER - 1] = '\0';
    strncpy(slot->subject, subject, MAX_SUBJECT - 1);
    slot->subject[MAX_SUBJECT - 1] = '\0';
    strncpy(slot->date, date, MAX_DATE - 1);
    slot->date[MAX_DATE - 1] = '\0';
    slot->categoryRank = categoryToRank(sender);
    slot->dateValue = dateToValue(date);

    heap->size++;
    siftUp(heap, heap->size - 1);
}

/* =====
* Peek at the highest priority email without removing it.
* Returns 1 on success (and fills 'out'), 0 if heap is empty.
* ===== */
static int heapPeek(MaxHeap *heap, Email *out) {
    if (heap->size == 0) {
        return 0;
    }
    *out = heap->data[0];
    return 1;
}

/* =====
* Remove the highest priority email from the heap.
* Returns 1 on success, 0 if the heap was already empty.
* ===== */
static int heapExtractMax(MaxHeap *heap) {
    if (heap->size == 0) {
        return 0;
    }
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    if (heap->size > 0) {
        siftDown(heap, 0);
    }
    return 1;
}

```

```

/* =====
 * Command handling
 * ===== */

/* Parses "EMAIL <category>,<subject>,<date>" (the part AFTER
 * the literal "EMAIL " keyword has already been isolated in
 * 'rest') and inserts the resulting email into the heap. */
static void handleEmailCommand(MaxHeap *heap, char *rest) {
    char *firstComma = strchr(rest, ',');
    char *lastComma = strchr(rest, ',');

    if (firstComma == NULL || lastComma == NULL || firstComma == lastComma) {
        fprintf(stderr, "Warning: malformed EMAIL command ignored: %s\n", rest);
        return;
    }

    /* Category = text before the first comma */
    size_t categoryLen = (size_t)(firstComma - rest);
    char category[MAX_SENDER];
    if (categoryLen >= MAX_SENDER) categoryLen = MAX_SENDER - 1;
    strncpy(category, rest, categoryLen);
    category[categoryLen] = '\0';

    /* Date = text after the last comma */
    char date[MAX_DATE];
    strncpy(date, lastComma + 1, MAX_DATE - 1);
    date[MAX_DATE - 1] = '\0';

    /* Subject = everything between the first and last comma */
    size_t subjectLen = (size_t)(lastComma - (firstComma + 1));
    char subject[MAX_SUBJECT];
    if (subjectLen >= MAX_SUBJECT) subjectLen = MAX_SUBJECT - 1;
    strncpy(subject, firstComma + 1, subjectLen);
    subject[subjectLen] = '\0';

    heapInsert(heap, category, subject, date);
}

static void handleNextCommand(MaxHeap *heap) {
    Email top;
    if (!heapPeek(heap, &top)) {
        printf("\nNo emails to read.\n");
        return;
    }
    printf("\nNext email:\n");
    printf("\tSender: %s\n", top.sender);
    printf("\tSubject: %s\n", top.subject);
    printf("\tDate: %s\n", top.date);
}

```

```

static void handleReadCommand(MaxHeap *heap) {
    /* Removes the highest priority email without displaying it.
    * If the queue is already empty, do nothing. */
    heapExtractMax(heap);
}

static void handleCountCommand(MaxHeap *heap) {
    if (heap->size == 1) {
        printf("There is 1 email to read.\n");
    } else {
        printf("There are %d emails to read.\n", heap->size);
    }
}

/* =====
* Main: reads commands line by line from stdin.
* ===== */
int main(void) {
    MaxHeap heap;
    heapInit(&heap);

    char line[MAX_LINE];

    while (fgets(line, sizeof(line), stdin) != NULL) {
        trimNewline(line);

        /* Skip blank lines */
        if (line[0] == '\0') {
            continue;
        }

        if (strncmp(line, "EMAIL ", 6) == 0) {
            handleEmailCommand(&heap, line + 6);
        } else if (strcmp(line, "NEXT") == 0) {
            handleNextCommand(&heap);
        } else if (strcmp(line, "READ") == 0) {
            handleReadCommand(&heap);
        } else if (strcmp(line, "COUNT") == 0) {
            handleCountCommand(&heap);
        } else {
            fprintf(stderr, "Warning: unrecognized command ignored: %s\n", line);
        }
    }

    heapFree(&heap);
    return 0;
}

```

GEMINI - Raw Generated Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE_LEN 256
#define MAX_SUBJECT_LEN 128
#define MAX_CATEGORY_LEN 32

typedef struct {
    char sender[MAX_CATEGORY_LEN];
    char subject[MAX_SUBJECT_LEN];
    char date[11]; // MM-DD-YYYY\0
    int category_rank;
    int year;
    int month;
    int day;
} Email;

typedef struct {
    Email *arr;
    int capacity;
    int size;
} MaxHeap;

// Returns priority score for sender category (higher number = higher priority)
int get_category_rank(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

// Parses date MM-DD-YYYY into integer components
void parse_date(const char *date, int *month, int *day, int *year) {
    sscanf(date, "%d-%d-%d", month, day, year);
}

// Compares two emails. Returns positive if a > b, negative if a < b, 0 if equal.
int compare_emails(const Email *a, const Email *b) {
    // Primary criteria: Sender category priority
    if (a->category_rank != b->category_rank) {
        return a->category_rank - b->category_rank;
    }
    // Secondary criteria: Date (Newer date has higher priority)
```

```

    if (a->year != b->year) return a->year - b->year;
    if (a->month != b->month) return a->month - b->month;
    return a->day - b->day;
}

```

// Initializes heap

```

MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = initial_capacity;
    heap->size = 0;
    heap->arr = (Email*)malloc(sizeof(Email) * heap->capacity);
    return heap;
}

```

```

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

// Maintains MaxHeap property upward

```

void heapify_up(MaxHeap *heap, int idx) {
    while (idx > 0) {
        int parent = (idx - 1) / 2;
        if (compare_emails(&heap->arr[idx], &heap->arr[parent]) > 0) {
            swap(&heap->arr[idx], &heap->arr[parent]);
            idx = parent;
        } else {
            break;
        }
    }
}

```

// Maintains MaxHeap property downward

```

void heapify_down(MaxHeap *heap, int idx) {
    int largest = idx;
    while (1) {
        int left = 2 * idx + 1;
        int right = 2 * idx + 2;

        if (left < heap->size && compare_emails(&heap->arr[left], &heap->arr[largest]) > 0) {
            largest = left;
        }
        if (right < heap->size && compare_emails(&heap->arr[right], &heap->arr[largest]) > 0) {
            largest = right;
        }

        if (largest != idx) {
            swap(&heap->arr[idx], &heap->arr[largest]);
        }
    }
}

```

```

        idx = largest;
    } else {
        break;
    }
}
}

// Dynamic array insertion into heap
void insert_email(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->arr = (Email*)realloc(heap->arr, sizeof(Email) * heap->capacity);
    }
    heap->arr[heap->size] = email;
    heap->size++;
    heapify_up(heap, heap->size - 1);
}

// Deletes and removes top priority email
void delete_max(MaxHeap *heap) {
    if (heap->size == 0) return;
    heap->arr[0] = heap->arr[heap->size - 1];
    heap->size--;
    if (heap->size > 0) {
        heapify_down(heap, 0);
    }
}

// Frees heap allocation
void free_heap(MaxHeap *heap) {
    free(heap->arr);
    free(heap);
}

int main() {
    MaxHeap *heap = create_heap(10);
    char line[MAX_LINE_LEN];

    while (fgets(line, sizeof(line), stdin)) {
        // Strip trailing newline character
        line[strcspn(line, "\r\n")] = 0;

        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            char *data = line + 6;
            char category[MAX_CATEGORY_LEN];
            char subject[MAX_SUBJECT_LEN];
            char date[11];

```

```

// Parse formatted strings delimited by commas
char *token1 = strtok(data, ",");
char *token2 = strtok(NULL, ",");
char *token3 = strtok(NULL, ",");

if (token1 && token2 && token3) {
    strncpy(category, token1, sizeof(category));
    strncpy(subject, token2, sizeof(subject));
    strncpy(date, token3, sizeof(date));

    Email email;
    strncpy(email.sender, category, sizeof(email.sender));
    strncpy(email.subject, subject, sizeof(email.subject));
    strncpy(email.date, date, sizeof(email.date));
    email.category_rank = get_category_rank(category);
    parse_date(date, &email.month, &email.day, &email.year);

    insert_email(heap, email);
}
} else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap->size);
} else if (strcmp(line, "NEXT") == 0) {
    if (heap->size > 0) {
        Email top = heap->arr[0];
        printf("\nNext email:\n");
        printf("\tSender: %s\n", top.sender);
        printf("\tSubject: %s\n", top.subject);
        printf("\tDate: %s\n\n", top.date);
    }
} else if (strcmp(line, "READ") == 0) {
    if (heap->size > 0) {
        delete_max(heap);
    }
}
}

free_heap(heap);
return 0;
}

```

e) The following is the analysis and comparison of the two GenAI programs:

Correctness:

Both programs that Claude and Gemini generated are mostly correct. Both compile successfully and correctly handle the main requirements of the prompt. They both successfully prioritize the emails based on sender category and date. They also both correctly handle various different test cases and edge cases. Some examples are two NEXT commands in a row, two READ

commands in a row, and commands when there are no emails in the heap. One slight issue with Gemini is that it prints an extra blank line after each email, while Claude only has one, more closely matching the guidelines of the assignment. Another issue that Gemini has is its handling of longer subject lines. Gemini has a buffer of 128 characters and doesn't always terminate the string properly if the subject fills the buffer. When a 150-character subject was tested, Gemini wrongly included the date at the end of the subject while Claude handled it correctly.

Execution Time:

The execution time for both Claude and Gemini is very similar; they both utilize the MaxHeap in basically the same way. For both, a normal heap insertion is $O(\log n)$ because the email may have to move up the heap. READ is also $O(\log n)$ for both programs because if the highest-priority email is removed, a different email may have to move down the heap to maintain the correct order. In both programs, NEXT and COUNT are $O(1)$ because they directly access a value without having to iterate through the heap. They directly access the highest-priority email or the current size of the heap. Both programs have the capability to resize the heap should it become full. Resizing the heap can have a complexity of $O(n)$ for that specific action, but this doesn't happen on every insertion because the capacity gets doubled with each resize. Overall, both have very similar execution-time complexity.

Space Complexity:

Both Claude and Gemini have an identical overall space complexity of $O(n)$ due to the amount of memory growing depending on the number of emails in the MaxHeap. The main difference between the two is the amount of memory used per email. Claude allocates more memory for each email, using a 64-character buffer for the sender, a 512-character buffer for the subject, and a 16-character buffer for the date. Gemini uses buffer sizes of 32 characters for the sender, 128 for the subject, and 11 for the date. This means that Gemini uses less total memory per email, but Claude is able to handle longer subject lines, which could be a benefit. Also, since both programs are able to resize the heap when needed, the amount of allocated memory can grow as emails keep getting added. Both programs still have an overall space complexity of $O(n)$, even with Gemini using less memory per email.

Maintainability:

Both programs use separate functions to manage MaxHeap operations and overall have pretty good maintainability, but there are some differences between the two. Gemini's program is overall shorter, which makes it slightly easier to follow. However, Claude separates more of the program into individual functions, which makes specific parts of the program easier to change without possibly affecting other parts. An example of this is how Claude has a separate function for handling and parsing EMAIL commands, while Gemini does this inside of main. Claude also has more error checking and handling, like checking to see if memory allocation was successful and checking for EMAIL command formatting issues. Gemini doesn't perform as many checks like this. Another difference is the way both programs handle dates. Gemini separates the date into year, month, and day and compares each individually. Claude converts each date into a

single numerical value for comparisons. Overall, Claude has more safeguards and separates different parts of the program better, while Gemini's benefit is that it is shorter.

f) Program selection and justification: Claude

I chose to work with Claude's version of the program as the basis for the assignment. Gemini and Claude had very similar time complexities, and Gemini had the benefit of using less memory per email, but Claude was stronger in correctness and maintainability. Claude was able to handle the longer subject line correctly while Gemini could not. Claude also had better formatting that matched the expected output more closely. Another benefit was that Claude separated the program into more individual functions, which made the code easier to understand and change. The error checking, such as checking memory allocation and EMAIL commands, was also a strong advantage. Overall, I decided Claude was the better option due to its stronger correctness and error handling without having a worse overall time complexity.

g) The following is a summary of the improvements made to the code

Correctness:

A change related to correctness and output consistency was made to the way the NEXT command is handled when no emails are in the heap. The original code printed an extra blank line before the message saying that there were no emails. This was removed to improve formatting of the messages. I also tested the revised program using rigorous test cases. Edge cases, priority-order tests, and heap resizing tests with more than 16 emails were run to verify correctness and functionality. The edge-case tests also confirmed that two NEXT commands in a row showed the same email, consecutive READ commands removed the correct emails, and commands on an empty heap didn't cause the program to crash. The program was able to continue to correctly prioritize emails by category, then by date after changes were made. Before accessing the top email, the empty-heap handling is still checked.

Execution Time:

An improvement made was changing heapPeek() so that it returns a pointer to the highest priority email instead of copying the entire Email structure into another variable. The NEXT command's time complexity remains the same with this change, but the amount of unnecessary copying is reduced whenever NEXT is called. Although the time complexity of NEXT stays at $O(1)$, this change helps reduce the constant amount of work being done since the entire Email structure doesn't have to be fully copied anymore.

Space Complexity:

An improvement made to space complexity was reducing the amount of memory reserved for some fields inside each Email. MAX_SENDER was reduced from 64 characters to 16 because the longest possible sender category only needs 15 characters plus the null terminator, that being the ImportantPerson category. MAX_DATE was reduced from 16 to 11 because the date

in MM-DD-YYYY only needs 10 characters plus the null terminator. dateValue was also changed from a long to an int because the YYYYMMDD value can fit inside an integer. These changes don't change the overall space complexity of $O(n)$, but they reduce the amount of memory needed for each Email in the heap. Since every email in the heap contains these fields, the memory savings increase the more emails are added, even though the overall space complexity stays at $O(n)$.

Maintainability:

The first improvement for maintainability was removing the ctype.h library since nothing in the program required it and it was unused. I also added const to pointers that should only be used for reading. For example, handleEmailCommand() now treats the command as read-only; handleNextCommand() and handleCountCommand() use read-only heap pointers since those commands shouldn't be modifying the heap. This helps prevent accidental changes to data in the code and allows the compiler to catch accidental attempts to modify read-only data. The return value from heapExtractMax() was also removed. The return value from this function was unused so removing it made the function simpler. Detailed comments were also added, as well as attributions labeling where code came from and who authored it. These changes overall make the program easier to follow, read, and maintain.

Other GenAI Assistance: ChatGPT was used for assistance with prompting, analyzing the generated programs, making improvements, and adding comments.